

Software Supply Chain Attacks and Build Provenance

軟體供應鏈攻擊與建置來源證明

NG, Chun Sing (r14922186) OU, Chia-Yun (r14922072)

May 31, 2026

Abstract

現代軟體開發高度依賴開源套件、套件管理器、版本控制平台、持續整合與持續部署（CI/CD）流程，以及第三方套件註冊庫。這些機制使軟體重用與自動化發布更加便利，卻也使開發流程建立在複雜的信任關係之上。開發者在日常工作中執行 `npm install`、`pip install`、`mvn install` 或 `conda install` 時，實際上不只是下載程式碼，也可能間接信任大量 `transitive dependencies`、維護者帳號、註冊庫 `metadata`、安裝腳本、建置流程與發布管線。

本報告探討軟體供應鏈攻擊（`software supply chain attacks`）與建置來源證明（`build provenance`）在防禦此類攻擊時的角色。本文首先說明軟體供應鏈安全的定義、風險上升的原因，以及其在 OWASP Top 10:2025 中被列為 A03 Software Supply Chain Failures 的背景。接著，本文分析 Axios 與 TanStack 兩個 `npm` 生態系事件。Axios 事件顯示，攻擊者可透過被竊取的 `maintainer credentials`、惡意套件發布、隱藏相依套件與 `postinstall` 腳本，在一般安裝流程中植入惡意程式。TanStack 事件則進一步指出，即使 `artifact` 具有有效的 `build provenance`，只要受信任的 CI/CD workflow 或 `cache trust boundary` 遭到攻破，`provenance` 仍可能無法阻止惡意套件被合法流程發布。

本文的核心論點是：`build provenance` 是必要的，但不足以單獨構成完整防禦。Provenance 能夠提高 `artifact traceability`，使 `source commit`、`builder identity`、`build workflow` 與 `artifact hash` 之間的關係更可驗證；然而，它並不能證明 `artifact` 本身必然良性，也不能保證建置環境未被入侵。因此，有效的軟體供應鏈防禦應採取分層策略，結合 `dependency scanning`、`lockfile discipline`、`install-time verification`、`maintainer account protection`、`CI/CD isolation`、`cache hardening`、`credential restriction`、`signed attestations` 與 `provenance verification`。

關鍵字： `software supply chain security`, `build provenance`, `SLSA`, `Sigstore`, `npm`, `CI/CD security`, `dependency attack`

目錄

1 Introduction	4
2 Background: Software Supply Chain Security	4
2.1 Definition of Software Supply Chain	4
2.2 Representative Forms of Software Supply Chain Attacks	5
2.3 Why Software Supply Chain Failure Became a Top Risk	6
2.4 Economic and Ecosystem Impact	6
3 Threat Model and Attack Surfaces	7
4 Case Study I: Axios npm Compromise	7
4.1 Incident Overview	7
4.2 Attack Flow	8
4.3 Limitations of Common Defenses	8
5 The Core Gap: No Tamper-Evident Chain	9
6 Build Provenance as a Defense Mechanism	9
6.1 Concept of Build Provenance	9
6.2 What Provenance Can Prove	10
6.3 Verification: Turning the Record into Prevention	10
6.4 Technology Stack and Workflow	11
6.5 Adoption Status	11
7 Case Study II: TanStack npm Supply-Chain Compromise	12
7.1 Incident Overview	12
7.2 How the Attack Worked	13
7.3 Why This Challenges Provenance	13
7.4 Industry Response: OpenAI	14
8 Toward a Layered Defense Strategy	14
8.1 Dependency Layer	14
8.2 Source Layer	14
8.3 Build Layer	15
8.4 Release Layer	15
9 Discussion	15
9.1 Practical Value of Build Provenance	15
9.2 Fundamental Limitation of Build Provenance	15
9.3 Synthesis of the Two Case Studies	16

10 Conclusion

16

1 Introduction

現代軟體開發已不再是一個完全由單一開發團隊掌控的封閉流程。開發者經常透過套件管理器取得現成函式庫，並將建置、測試、發布與部署交給自動化工具執行。例如，日常開發中常見的指令如下：

```
npm install    pip install    mvn install    conda install
```

這些指令在工程上提高了效率，因為團隊可以快速重用已存在的軟體元件。然而，這種便利性同時擴大了攻擊面。安裝一個直接相依套件（**direct dependency**）時，套件管理器可能同時解析並下載數十個 **transitive packages**；在某些生態系中，安裝過程也可能執行 **preinstall**、**install** 或 **postinstall** 等 **lifecycle scripts**。換言之，開發者表面上只執行一個安裝指令，實際上卻信任了許多未經人工逐一檢查的上游元件與中介流程。

軟體供應鏈攻擊的危險性正在於此。攻擊者不一定需要直接攻破最終應用程式，而是可以攻擊已被開發者信任的任一環節，例如 **dependency package**、**maintainer account**、**package registry**、**build cache**、**CI/CD workflow** 或 **release pipeline**。一旦受信任的上游環節被攻破，惡意程式碼就可能透過正常安裝或正常發布流程進入大量 **downstream projects**。

本文聚焦的研究問題是：**build provenance** 是否能有效防止軟體供應鏈攻擊？本報告採取較謹慎的立場。**Build provenance** 可以讓 **artifact** 的來源、建置流程與發布紀錄變得更可追蹤，也能提供 **install-time verification** 的基礎；但是，如果合法 **workflow** 本身已被攻擊者利用，**provenance** 仍可能產生一份「正確但不充分」的紀錄。因此，本報告將 **provenance** 視為 **defense-in-depth** 中的關鍵機制，而非單一完整解法。

2 Background: Software Supply Chain Security

2.1 Definition of Software Supply Chain

根據 NIST 的說明，**software supply chain** 是一組用來建立、轉換並評估軟體 **artifacts** 品質與政策符合性的步驟。在實務上，這條供應鏈涵蓋 **source code**、**third-party dependencies**、**version control systems**、**build tools**、**CI/CD pipelines**、**package registries**、**artifact repositories**、**release workflows** 與 **deployment mechanisms**。也就是說，現代軟體產品的安全性不只取決於開發團隊撰寫的程式碼，也取決於其所依賴的整個開發、建置與發布環境。

Software supply chain attack 則是指攻擊者攻破或操控供應鏈中的某個受信任

節點。該節點可能是 `dependency package`、`maintainer account`、`build cache`、`CI/CD workflow`、`registry account`、`signing process` 或 `release pipeline`。攻擊者的目標不是單純利用應用程式本身的漏洞，而是濫用開發者、工具、套件與註冊庫之間既有的信任關係。由於熱門套件通常被大量專案重複使用，單一 `upstream compromise` 便可能造成大規模 `downstream impact`。

2.2 Representative Forms of Software Supply Chain Attacks

為了說明 `software supply chain attacks` 的攻擊面並不侷限於單一套件漏洞，本節整理六種具有代表性的攻擊形式。相關研究曾將 `SSC` 攻擊整理成更完整的 `taxonomy`，涵蓋名稱偽裝、套件接管、帳號濫用、`registry` 行為與建置流程等多種攻擊途徑 [27]。本文僅選取與本簡報主軸最相關的六類，用來銜接後續 `Axios` 與 `TanStack` 兩個案例。

表 1: 代表性的 `software supply chain attack forms`

Attack Form	Description
Typosquatting	攻擊者發布名稱與熱門套件相似的惡意套件，利用拼字錯誤或近似命名誘使開發者誤裝。
Dependency Confusion	當專案同時使用 <code>public registry</code> 與 <code>private registry</code> 時，攻擊者在 <code>public registry</code> 發布同名套件，使 <code>package manager</code> 解析到錯誤來源。
Account Takeover	攻擊者竊取 <code>maintainer</code> 或 <code>developer</code> 帳號憑證，接著使用合法帳號權限發布惡意套件版本。
Package Takeover	攻擊者取得無人維護、轉移所有權或信任邊界鬆散的套件控制權，再發布帶有惡意內容的新版本。
Code Injection	攻擊者將惡意程式碼注入到原本受信任的 <code>package</code> 、 <code>dependency</code> 或 <code>release artifact</code> 中，使惡意行為混入正常軟體流程。
CI/CD Compromise	攻擊者濫用 <code>build workflows</code> 、 <code>cache</code> 、 <code>runner environment</code> 或 <code>tokens</code> ，使合法的建置與發布流程產出惡意 <code>artifact</code> 。

這些攻擊形式說明，`software supply chain security` 不只是在檢查 `source code` 本身，也必須涵蓋套件名稱、`maintainer accounts`、`dependency resolution`、`package registries`、`build pipelines` 與 `release process`。後續 `Axios` 事件主要展示 `account takeover`、`malicious package publication` 與 `installation script abuse` 的風險；`TanStack` 事件則進一步展示 `CI/CD compromise` 與 `cache trust boundary` 的問題。

2.3 Why Software Supply Chain Failure Became a Top Risk

OWASP Top 10:2025 將 *Software Supply Chain Failures* 排為 A03，顯示應用程式安全的風險重心正在轉變。傳統應用程式安全多半聚焦於應用程式內部缺陷，例如 injection、authentication failure、insecure design 或 access control failure。相較之下，software supply chain security 更關注應用程式是如何被建置、依賴哪些外部元件、由誰發布這些元件，以及最終 artifact 是否能被驗證。

OWASP Top 10:2025 並非完全由單一統計資料直接決定，而是 data-informed。其排名結合 contributed testing data、CVE-based exploit and impact scores，以及 community survey results。對於 A03:2025 Software Supply Chain Failures，OWASP 報告的數據包括 5.72% average incidence rate、215,248 total occurrences，以及 11 個 mapped CVEs。這些數字應被解讀為目前測試資料與工具所能觀察到的風險表現，而不能直接等同於全球實際攻擊頻率。事實上，供應鏈攻擊往往涉及新發布惡意套件、憑證濫用或 CI/CD 流程被操控，這些情境未必會立即出現在 CVE 資料庫或傳統掃描結果中，因此真實風險可能被低估。

此問題之所以成為高風險議題，主要來自幾個結構性因素。第一，現代應用程式通常具有大型 dependency graph，少數 direct dependencies 可能帶入大量 transitive dependencies。第二，套件管理器高度自動化，會自動解析版本、下載套件、讀取 metadata，並在某些情況下執行安裝腳本。第三，CI/CD 已成為建置與發布的核心流程，使 workflow permissions、runner environment、OIDC tokens 與 build cache 成為重要攻擊面。第四，開發者通常不會人工審查每一個 package version、tarball、build log、artifact hash 與 provenance record。這些因素共同造成攻擊者得以透過一個受信任節點影響大量 downstream systems。

2.4 Economic and Ecosystem Impact

供應鏈攻擊的財務影響難以完全量化，因為許多事件不會公開揭露損失細節。然而，breach-cost data 仍顯示 supply chain compromise 的代價相當高。IBM 2025 breach-cost data 指出，third-party vendor and supply chain compromise 平均每次攻擊成本為 USD 4.91M，並被列為第二常見且第二高成本的 data breach vector。此類成本可能包括 incident response、downtime、credential rotation、CI/CD rebuild、legal risk 與 reputational damage。

同時，open-source package risk 也持續上升。ReversingLabs 2026 software supply chain security report 指出，2025 年 malicious open-source package detections 增加 73%，其中 npm 約佔 detected OSS malware 的 90%。這項趨勢說明，攻擊者已將 package ecosystems 視為具高度投資報酬率的攻擊面。npm 生態系尤其值得關注，因為它具有龐大的套件規模、快速的版本發布節奏、自動

化安裝流程，以及高度 transitive 的 dependency structure。

3 Threat Model and Attack Surfaces

軟體供應鏈攻擊可以分為 dependency layer、source layer、build layer 與 release layer 四個層面觀察。每個層面具有不同的 attack surface，也需要不同類型的防禦控制。

表 2: 軟體供應鏈中的主要攻擊面

Layer	Possible Attack Surface
Dependency	Malicious packages、typosquatting、dependency confusion、transitive dependency compromise，以及濫用 preinstall、install、postinstall lifecycle scripts。
Source	Compromised maintainer accounts、malicious commits、review bypass、弱化的 protected branch settings，以及過度授權的 release permissions。
Build	CI/CD workflow tampering、build cache poisoning、從 runner memory 擷取 token、不安全的 fork pull request workflows，以及 overprivileged OIDC tokens。
Release	Registry account compromise、artifact substitution、missing signatures、missing attestations, 以及缺少 provenance verification。

在 npm 生態系中，lifecycle scripts 是特別重要的攻擊面。preinstall、install 與 postinstall 可能在套件安裝階段自動執行。因此，攻擊者若能發布含有惡意 lifecycle script 的套件版本，受害者可能在 application import 該套件之前就已經執行惡意程式碼。這使 installation phase 本身成為供應鏈攻擊的重要 execution vector。

4 Case Study I: Axios npm Compromise

4.1 Incident Overview

Axios 是 JavaScript 生態系中廣泛使用的 HTTP client。根據簡報資料，Axios 每週約有 100 million downloads。2026 年 3 月 31 日，Axios 發生 npm supply chain compromise。攻擊者使用被竊取的 maintainer credentials 發布惡意套件版本，暴露時間約為三小時。惡意版本注入一個 attacker-controlled hidden dependency，並透過 postinstall script 在正常安裝過程中安裝 Remote Access Trojan(RAT)。

此事件具有代表性，因為攻擊並不依賴使用者主動執行應用程式。使用者只要執行 npm install axios，安裝流程就可能觸發惡意腳本。換言之，攻擊發生於開發與安

裝階段，而不是部署後的 runtime 階段。這種攻擊方式也使傳統 application-level security controls 較難觀察到最初的 compromise path。

根據簡報內容，此事件也影響 OpenAI 的 macOS app-signing pipeline，使 ChatGPT Desktop 與 Codex 的 code-signing certificates 面臨風險。這一點凸顯供應鏈攻擊的放大效應：當受影響者不是單一終端使用者，而是擁有簽章憑證、建置權限或發布權限的開發環境時，攻擊後果可能進一步擴散到更多 downstream users。

4.2 Attack Flow

Axios 事件可概念化為下列攻擊流程：

```
npm install axios
-> malicious axios version
    -> attacker-controlled hidden dependency
        -> postinstall script executes
            -> Remote Access Trojan installed silently
```

此流程的危險性在於它與正常開發行為高度相似。套件名稱、registry entry 與安裝指令都可能看起來正常，但實際 tarball 與 dependency graph 已被攻擊者操控。由於沒有明顯 prompt 或 manual review，受害者很可能在毫無察覺的情況下被 compromise。

4.3 Limitations of Common Defenses

Axios 事件也揭示常見 dependency defenses 的限制。npm audit 與 Snyk 對已知 CVEs 很有價值，但新發布的 malicious version 可能尚未被登錄於 vulnerability database。npm ci 與 lockfile 能夠改善 reproducibility，並驗證 registry 中的 pinned versions 與 hash；然而，如果 lockfile 是在惡意版本可用期間產生，它可能反而固定 compromised version。Dependabot 則能自動更新已知 vulnerable dependencies，但其判斷仍高度依賴 CVE 與 vulnerability feeds；自動升級也可能意外引入剛被攻破的新版本。

表 3: 常見防禦方式與其限制

Defense	Protection	Limitation
npm audit / Snyk	偵測 dependency tree 中已知 CVEs 與已知 vulnerable dependencies。	對 newly published malicious versions 反應較慢，因為惡意版本可能尚未被分配 CVE 或收錄進資料庫。

Defense	Protection	Limitation
npm ci + lockfile	對 pinned versions 與 registry hash 進行驗證，提高 build reproducibility。	仍然信任 lockfile；若 lockfile 於攻擊暴露期間產生，可能鎖定惡意版本。
Dependabot	自動更新已知 vulnerable dependencies。	仍依賴 vulnerability database；更新本身也可能引入新風險。

因此，供應鏈防禦不能只依靠 known-vulnerability scanning。對於惡意套件發布、憑證竊取與安裝腳本濫用，防禦也必須納入 package metadata、publishing behavior、lifecycle scripts、provenance status 與 install-time risk checks。

5 The Core Gap: No Tamper-Evident Chain

Axios 事件反映一個更根本的問題：在 source code、build system、registry package 與 final installation 之間，傳統套件安裝流程未必形成可驗證的 tamper-evident chain。一般開發者在安裝套件時，通常無法清楚回答以下問題：

- Artifact 是否由預期的 source commit 建置而來？
- Build steps 是否未被修改或濫用？
- Registry 中發布的 tarball 是否與 CI output 相符？
- Artifact 是否由預期的 workflow、builder identity 與 release process 產生？

當這些問題無法被回答時，攻擊可能維持沉默。套件的名稱、版本號與 registry 頁面仍然可信，但 artifact 本身未必來自預期來源。Build provenance 的主要價值，就是試圖在 source 與 install 之間建立可簽署、可驗證、可追蹤的證據鏈。

6 Build Provenance as a Defense Mechanism

6.1 Concept of Build Provenance

Build provenance 是描述 software artifact 如何被產生的 metadata。它通常記錄 source repository、commit hash、builder identity、build workflow、build

time、build type 與 artifact hash。簡報中以一句話概括其意義: *Built from commit X, by pipeline Y, at time T*。若這份紀錄經由簽署並寫入 transparency log, consumer 便能在安裝或驗證時檢查 artifact 是否符合預期。

在 SLSA 與 Sigstore 的模型中, build provenance 通常結合 OIDC identity、keyless signing、artifact hashes 與 Rekor public append-only log。SLSA 提供供應鏈完整性框架, 說明 provenance 應證明什麼; Sigstore 則提供簽署與 transparency log infrastructure, 使 provenance statement 更具可驗證性。這些機制的共同目標不是單純產生 metadata, 而是使 source-to-artifact path 更具 tamper-evident 特性。

6.2 What Provenance Can Prove

Build provenance 可以協助回答三類核心問題。第一, 它可透過 signed commit hash 記錄 artifact 是否來自預期 source commit。第二, 它可透過 OIDC-bound builder identity 連結 artifact 與特定 build pipeline。第三, 它可透過 artifact hash 檢查 registry tarball 是否與 attested CI output 一致。

表 4: Build provenance 可以提供的驗證資訊

Question	How Provenance Helps
Correct source commit?	Commit hash 會被記錄並簽署, 使 consumer 能檢查 artifact 是否來自預期 source revision。
Build steps unmodified?	Pipeline identity 可透過 OIDC 與 attestation 綁定, 使 consumer 能檢查 builder 是否符合預期。
Tarball matches CI output?	Artifact hash 可被記錄在 provenance statement 或 transparency log 中, 使 registry artifact 可與 CI output 比對。

因此, provenance 的核心貢獻在於 traceability。Consumer 不必只依賴 package name 與 version number, 而是可以要求 artifact 提供來源、建置流程與簽章證據。

6.3 Verification: Turning the Record into Prevention

Provenance 本身是一份紀錄; 若要將其轉化為預防性控制, 必須在 install time 或 policy enforcement 階段進行驗證。SLSA 的 artifact verification 模型可簡化為三個步驟。第一, consumer 驗證 provenance signature, 確認 attestation 未被竄改。第二, consumer 將 provenance 中的 source repository、builder identity 與 build type 與 expected values 比較。第三, 在需要更高保證的情境下, consumer 可遞迴驗證 dependencies。

Expected values 可以由 maintainer 在 repository 中宣告，也可以由 organization policy 指定，或由 first observed attestation 學習而來。這個設計使 provenance 不只是被動紀錄，而是可以成為 install-time gate。例如，如果某 package 先前版本沒有 attestation，工具可以發出 warning；如果先前版本有 attestation，但目標版本突然缺少 attestation，工具可以 raise error 或阻止安裝；若目標版本有 attestation，工具則可以驗證 signature 並繼續安裝流程。

6.4 Technology Stack and Workflow

實務上，build provenance 涉及多個技術層次。SLSA 定義 provenance 與 build levels 所需證明的內容；Sigstore 提供 keyless signing 與 Rekor transparency log；registry 與 CI platform 負責產生、儲存與發布 attestations；consumer-side tools 如 npq 或 Socket Firewall 則可作為安裝前的檢查機制，用於 provenance verification，並輔助判斷套件安裝風險。

表 5: Build provenance 相關技術層次

Layer	Technology	Role
Framework	SLSA	定義 provenance 應證明的內容，以及 build integrity levels。
Cryptography	Sigstore	提供 keyless signing 與 public transparency log support，包括 Rekor。
Platform	Registry + CI	產生、儲存與發布 attestations，例如 npm、PyPI 或 GitHub Actions。
Consumer	npq / Socket Firewall	作為 pre-install gate，用於 provenance verification，並輔助判斷套件安裝風險。

Producer side 的流程通常是在 CI/CD 中從特定 source commit 建置 artifact，產生 provenance statement，將 commit、builder、artifact hash 等資訊簽署並記錄，最後由 registry 儲存 package 與 attestation metadata。Consumer side 則是在安裝前取得 package 與 attestations，驗證 signature，檢查 expected source、builder 與 build type，並依政策決定 warning、blocking 或允許安裝。

6.5 Adoption Status

Provenance support 已逐漸出現在主要 package ecosystems 中，但採用仍屬早期階段。簡報資料整理指出，npm 的 npm publish --provenance 自 2023 年 10 月起 generally available；GitHub Artifact Attestations 自 2024 年 6 月起 generally available；PyPI 於 2024 年 11 月支援 digital attestations；Maven、RubyGems 與 Conda 也陸續開始支援或實驗相關機制。

表 6: Provenance-related mechanisms 的採用狀態

Ecosystem	Status	Adoption	Source Men- tioned in Slides
npm	2023 年 10 月起 generally avail- able。	beta 期間已有 3,800 packages; provenance- enabled packages 超過 500M down- loads。	GitHub Blog / Sig- store Blog
PyPI	2024 年 11 月起 generally avail- able。	到 2025 年底, 17% uploads included attestations。	PyPI Blog
GitHub At- testations	2024 年 6 月起 gen- erally available。	簡報未列出公開 adoption figure。	GitHub Blog
Maven	2025 年 1 月起 opt- in。	Early adopters only。	Sonatype
RubyGems	In progress。	2025 年 3 月有 20 top gems shipping attestations。	RubyGems Blog
Conda	2025 年 8 月起 beta。	簡報未列出公開 adoption figure。	prefix.dev

此採用狀態說明 provenance 正在逐漸成熟，但尚未形成普遍強制標準。主要限制不只在於工具是否支援，也包括 maintainer awareness、workflow migration overhead，以及 consumer 是否真的在 install time 驗證 provenance，而不是僅僅看到 attestation 存在就假設安全。

7 Case Study II: TanStack npm Supply-Chain Compromise

7.1 Incident Overview

TanStack 事件凸顯 build provenance 的邊界。根據簡報資料，TanStack 於 2026 年 5 月 11 日發生 npm supply-chain compromise。短時間內, 42 packages 與 84 malicious versions 被發布; payload 目標包括 cloud credentials、API tokens

與 SSH keys。此事件由外部研究者在 26 分鐘內偵測到。

此事件最值得注意之處在於：惡意版本具有 **valid build provenance**。也就是說，問題不在於 **provenance** 沒有產生，也不在於 **attestation** 與 **package** 完全不相符。相反地，**provenance** 正確記錄 **package** 由 TanStack 的合法 **workflow** 產生；真正的問題是合法 **workflow** 所依賴的 **cache** 與 **token trust boundary** 已被攻擊者濫用。

7.2 How the Attack Worked

簡報中的攻擊流程可概括如下：攻擊者透過 **fork pull request** 毒化 **shared CI cache**。當後續合法 **release workflow** 執行時，它 **restore** 了被毒化的 **cache**。被 **restore** 的 **cache** 接著從 **runner memory** 中擷取 **OIDC token**，並透過合法發布流程推出 **malicious packages**。簡化後的流程如下：

```
fork pull request
-> shared CI cache poisoned
    -> legitimate release workflow restores cache
        -> OIDC token extracted from runner memory
            -> malicious packages published through legitimate
                workflow
                    -> valid build provenance generated
```

此事件的關鍵在於 **trust boundary** 的錯置。**Fork pull request workflow** 屬於較低信任等級，而 **release workflow** 屬於高信任等級；若兩者共享 **cache** 或可互相影響，攻擊者便可能利用低信任入口污染高信任發布流程。這說明 **CI/CD cache**、**runner memory**、**OIDC token** 與 **workflow permissions** 都是 **provenance** 之外必須保護的攻擊面。

7.3 Why This Challenges Provenance

TanStack 事件證明了一個重要限制：**provenance proves origin, not innocence**。**Provenance** 可以證明 **artifact** 來自某個 **repository**、**workflow** 或 **builder**，但它無法單獨證明 **workflow** 執行時沒有被污染。若合法 **workflow** 本身被攻擊者利用，**provenance** 仍會準確記錄該 **artifact** 來自該 **workflow**，但 **artifact** 依然可能是惡意的。

因此，**build provenance** 是必要條件，而不是充分條件。它能使供應鏈更透明、更可追蹤，也能幫助 **consumer** 發現 **artifact substitution** 或 **missing attestation**。然而，它不能取代 **CI/CD isolation**、**cache hardening**、**credential restriction** 與 **workflow permission auditing**。換言之，**provenance** 提高了攻擊成本，但攻擊者仍可能透過破壞 **build environment** 來繞過 **provenance** 所提供的保證。

7.4 Industry Response: OpenAI

簡報也提到 OpenAI 對 TanStack 事件的回應。TanStack incident 中，有兩台 OpenAI employee devices 受到影響，而當時相關控制措施仍處於 phased roll-out。Axios 事件後已部署的控制措施包括：強化 CI/CD pipelines 中的 credential materials、設定 package manager 的 minimumReleaseAge 以拒絕剛發布的新版本，以及使用 security software validate provenance of new packages。

此回應與前述分析一致。Provenance verification 很重要，但組織仍需要降低「剛發布惡意版本」進入開發環境的機率，也需要保護 CI/CD 與開發設備中的 credentials。這些控制措施共同構成 layered defense，而非依賴單一機制解決所有風險。

8 Toward a Layered Defense Strategy

Axios 與 TanStack 兩個案例分別呈現不同層面的供應鏈風險。Axios 主要說明 maintainer credentials、malicious package publication 與 lifecycle scripts 的問題；TanStack 則說明 CI/CD cache、OIDC tokens 與 trusted workflow compromise 的問題。兩者共同指出：單一防禦機制無法覆蓋完整供應鏈攻擊面。因此，實務防禦必須採取分層策略。

8.1 Dependency Layer

Dependency layer 的目標是降低惡意或脆弱套件進入專案的機率。實務上，組織應使用 lockfiles 控制版本與 hash，並避免不必要的 dependency updates；同時，可使用 SBOMs 記錄 dependency composition，並整合 npm audit、OSV 或 Snyk 等 vulnerability scanners 以偵測已知 vulnerabilities。然而，這些工具不足以處理 newly published malicious packages，因此也應搭配 npq 或 Socket Firewall 等 install-time gates，檢查 package metadata、provenance status 與 suspicious behavior。對於高風險環境，minimum release age policy 也能降低剛發布惡意版本立即進入專案的機率。

8.2 Source Layer

Source layer 主要保護 source repositories 與 maintainer accounts。維護者帳號若被竊取，攻擊者可直接發布惡意版本或修改 release process。因此，multi-factor authentication、protected branches、mandatory code review 與 restricted publishing permissions 都是必要控制。除此之外，組織也應監控異常 commits、releases、token usage 與 maintainer activity，以便在受信任帳號行為異常時及早發現風險。

8.3 Build Layer

Build layer 是 TanStack 事件最重要的教訓。組織必須嚴格分離 untrusted pull request workflows 與 trusted release workflows，避免 fork PR workflows 與 release workflows 共用高信任 cache。GitHub Actions tokens、OIDC tokens 與 registry tokens 應遵循 least privilege principle，並避免 credential materials 出現在 runner memory、logs 或可被低信任流程讀取的位置。Cache restore paths、build scripts 與 release scripts 也應被視為安全邊界的一部分，而不是單純的效能最佳化工具。

8.4 Release Layer

Release layer 的目標是在 publication 與 installation 階段使 artifacts 可驗證。組織應使用 signed attestations 與 build provenance，將 source commit、builder identity、artifact hash 與 build workflow 記錄於可驗證 metadata 中。更重要的是，consumer 應在 install time 驗證 provenance，而不是只在 publish time 產生 provenance。若某 package 先前具有 attestation，但新版本突然缺少 attestation，安裝工具應至少 warning，甚至依政策 block installation。

9 Discussion

9.1 Practical Value of Build Provenance

Build provenance 的實務價值在於將原本隱性的信任關係轉化為可驗證的 evidence。過去，consumer 往往只能看到 package name、version number 與 registry metadata，卻難以確認 artifact 是否真的來自預期 source code 與 build process。Provenance 透過 signed attestations 建立 source、builder 與 artifact 之間的關係，使 artifact substitution、unauthorized publishing，以及部分 maintainer-compromise scenarios 更容易被偵測。

此外，provenance 也為 policy enforcement 提供基礎。企業或組織可以要求安裝的 packages 必須來自特定 repository、特定 builder 或具有有效 attestation。這使供應鏈安全從「相信 package 應該是正確的」轉為「要求 package 提供可機器驗證的證據」。從這個角度看，provenance 不只是紀錄工具，也是一種可被自動化安全政策利用的基礎建設。

9.2 Fundamental Limitation of Build Provenance

然而，provenance 不應被誤解為 malware scanner，也不應被視為完整的 build environment integrity proof。TanStack 事件說明，若合法 workflow 被污染，

provenance 可能仍然完全有效，但其所證明的是 artifact 的來源，而不是 artifact 的良性。換言之，provenance 可以回答「這個 artifact 從哪裡來」，但不能單獨回答「這個 artifact 是否安全」。

因此，較精確的結論是：**build provenance raises the bar, but the bar can still be cleared**。它提高攻擊者偽造 artifact origin 的難度，也使 missing provenance 或 unexpected builder 更容易被發現；但攻擊者仍可能透過 CI/CD cache poisoning、credential theft、workflow abuse 或 maintainer account compromise 繞過部分保證。Provenance 應被採用並驗證，但必須與 CI/CD isolation、cache hardening、credential restriction、dependency scanning 與 install-time controls 一起使用。

9.3 Synthesis of the Two Case Studies

Axios 與 TanStack 的差異有助於理解供應鏈安全的完整範圍。Axios 事件偏向 package publication 與 lifecycle script abuse：攻擊者利用被竊取的 maintainer credentials 發布惡意套件版本，並透過安裝腳本完成入侵。此情境中，provenance 與 install-time verification 可以增加使用者在安裝前發現異常的機會。

TanStack 事件則偏向 build pipeline compromise：攻擊者不是單純偽造 artifact origin，而是讓合法 workflow 產生惡意 artifact。此情境中，provenance 仍會產生看似正確的 attestation。因此，TanStack 事件提醒我們，build provenance 的可信度取決於 build environment 的可信度。若 build environment 的 trust boundary 設計不良，provenance 只能忠實記錄一個已被污染的流程。

10 Conclusion

Software supply chain attacks 之所以危險，是因為它們利用了現代軟體開發效率背後的信任結構。開發者信任套件管理器、套件維護者、註冊庫、CI/CD workflows、build caches 與 automated release processes。攻擊者則選擇攻擊這些受信任節點，使惡意程式碼能以正常開發活動的外觀進入 downstream systems。

Axios 事件顯示，被竊取的 maintainer credentials、malicious package publication、hidden dependencies 與 postinstall scripts 可以在正常安裝流程中完成入侵。TanStack 事件則進一步顯示，即使惡意套件具有 valid build provenance，只要 CI/CD workflow 或 shared cache 已被攻破，provenance 仍不足以阻止攻擊。兩個案例共同支持本文主張：build provenance 是必要的，但不是充分的。

Build provenance 的真正價值在於使 source、builder 與 artifact 之間的關係更可見、更可簽署、更可驗證。它可以支援 install-time verification 與 policy enforcement，並提高 artifact substitution 或 missing attestation 的可見度。然

而，它不能取代 **secure build environment**，也不能單獨證明 **artifact** 是良性的。因此，未來的軟體供應鏈防禦應將 **provenance** 視為基礎機制，並同時部署 **dependency scanning**、**lockfile discipline**、**minimum release age policy**、**maintainer account protection**、**CI/CD isolation**、**cache hardening**、**credential restriction** 與 **provenance verification**。只有透過這種 **layered defense**，才能較合理地降低現代軟體供應鏈攻擊的系統性風險。

參考文獻

- [1] National Institute of Standards and Technology. *NIST SP 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>
- [2] OWASP. *Software Supply Chain Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html
- [3] OWASP. *A03:2025 Software Supply Chain Failures*. https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/
- [4] OWASP. *OWASP Top 10:2025 Introduction and Methodology*. https://owasp.org/Top10/2025/0x00_2025-Introduction/
- [5] SLSA. *Supply Chain Threats*. <https://slsa.dev/spec/v1.0/threats-overview>
- [6] SLSA. *About SLSA*. <https://slsa.dev/spec/v1.2/about>
- [7] SLSA. *Verifying Artifacts*. <https://slsa.dev/spec/v1.0/verifying-artifacts>
- [8] OpenSSF. *Sigstore*. <https://www.sigstore.dev/>
- [9] Liran Tal. *npq: Safely install packages with npm or yarn by auditing them as part of your install process*. <https://github.com/lirantal/npq>
- [10] Socket. *Introducing Socket Firewall*. <https://socket.dev/blog/introducing-socket-firewall>
- [11] Axios. *Post Mortem: axios npm supply chain compromise*. <https://github.com/axios/axios/issues/10636>
- [12] Microsoft Security Blog. *Mitigating the Axios npm supply chain compromise*. <https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
- [13] OpenAI. *Our response to the Axios developer tool compromise*. <https://openai.com/index/axios-developer-tool-compromise>

- [14] TanStack. *Postmortem: TanStack npm supply-chain compromise.* <https://tanstack.com/blog/npm-supply-chain-compromise-postmortem>
- [15] TanStack. *Hardening TanStack After the npm Compromise.* <https://tanstack.com/blog/incident-followup>
- [16] OpenAI. *Our response to the TanStack npm supply chain attack.* <https://openai.com/index/our-response-to-the-tanstack-npm-supply-chain-attack>
- [17] GitHub Blog. *Artifact Attestations is generally available.* <https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/>
- [18] GitHub Blog. *Introducing npm package provenance.* <https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>
- [19] Sigstore Blog. *npm's Sigstore-powered provenance goes GA.* <https://blog.sigstore.dev/npm-provenance-ga/>
- [20] PyPI Blog. *PyPI now supports digital attestations.* <https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/>
- [21] PyPI Blog. *PyPI 2025 Year in Review.* <https://blog.pypi.org/posts/2025-12-31-pypi-2025-in-review/>
- [22] Sonatype Central. *Sigstore Signature Validation via Portal.* https://central.sonatype.org/news/20250128_sigstore_signature_validation_via_portal/
- [23] RubyGems Blog. *February 2025 Updates.* <https://blog.rubygems.org/2025/03/19/february-rubygems-updates.html>
- [24] prefix.dev. *Securing the Conda package supply chain with Sigstore.* <https://prefix.dev/blog/securing-the-conda-package-supply-chain-with-sigstore>
- [25] IBM. *Cost of a Data Breach Report 2025 / Attack vector overview.* <https://www.ibm.com/think/topics/attack-vector>

-
- [26] ReversingLabs. *Software Supply Chain Security Report 2026*. <https://www.reversinglabs.com/resources/software-supply-chain-security-report-2026>
- [27] ScienceDirect. *A taxonomy and analysis of software supply chain attacks*. <https://www.sciencedirect.com/science/article/pii/S2214212625003606>